



RISC-V Simulator Integer Multiplication and Division Extension

BY: CARLOS LUCERO

Instructions Included in M-Extension

Optional Multiply-Divide Instruction Extension: RVM					
Category	Name	Fmt	RV32M (Multiply-Divide)		+RV{64,128}
Multiply	MULTIPLY	R	MUL	rd,rs1,rs2	MUL{W D} rd,rs1,rs2
	MULTIPLY upper Half	R	MULH	rd,rs1,rs2	
	MULTIPLY Half Sign/Uns	R	MULHSU	rd,rs1,rs2	
	MULTIPLY upper Half Uns	R	MULHU	rd,rs1,rs2	
Divide	DIVide	R	DIV	rd,rs1,rs2	DIV{W D} rd,rs1,rs2
	DIVide Unsigned	R	DIVU	rd,rs1,rs2	
Remainder	REMAinder	R	REM	rd,rs1,rs2	REM{W D} rd,rs1,rs2
	REMAinder Unsigned	R	REMU	rd,rs1,rs2	REMU{W D} rd,rs1,rs2

- mul : Signed x Signed -> lower half
- mulh: Signed x Signed -> upper half
- mulhsu: Signed x Unsigned -> upper half
- mulhu: Unsigned x Unsigned -> upper half
- div / divu: Signed / Unsigned division
- Rem / remu: Signed / Unsigned modulo

What needs to be changed? / Considerations

- Only components that need changing are R-type Section of `alu_control` (Decoding new instructions correctly), `alu_exec` (Executing new instructions correctly) and `try_mnemonic` (Adding new instructions to trace).
- Considerations for multiplication:
 - Shift for 'high' variants
- Consideration for division / remainder:
 - Divide by Zero
 - Overflow when dividing int min by -1
 - Truncate results (RISC-V International, 2026)

Table 11. Semantics for division by zero and division overflow. L is the width of the operation in bits: $XLEN$ for $DIV[U]$ and $REM[U]$, or 32 for $DIV[U]W$ and $REM[U]W$.

Condition	Dividend	Divisor	$DIVU[W]$	$REMU[W]$	$DIV[W]$	$REM[W]$
Division by zero	x	0	2^{L-1}	x	-1	x
Overflow (signed only)	-2^{L-1}	-1	-	-	-2^{L-1}	0

```

217 def alu_control(c, d):
219     f3 = d["funct3"]
220     f7 = d["funct7"]
221
222     if op == "ADDR":
223         return "ADD"
224     if op == "BR":
225         return "SUB"
226
227     if op == "R":
228         if f3 == 0b000:
229             return "SUB" if f7 == 0b0100000 else ("MUL" if f7 == 0b0000001 else "ADD")
230         if f3 == 0b111:
231             return "REMU" if f7 == 0b0000001 else "AND"
232         if f3 == 0b110:
233             return "REM" if f7 == 0b0000001 else "OR"
234         if f3 == 0b100:
235             return "DIV" if f7 == 0b0000001 else "XOR"
236         if f3 == 0b001:
237             return "MULH" if f7 == 0b0000001 else "SLL"
238         if f3 == 0b101:
239             return "SRA" if f7 == 0b0100000 else ("DIVU" if f7 == 0b0000001 else "SRL")
240         if f3 == 0b010:
241             return "MULHSU" if f7 == 0b0000001 else "SLT"
242         if f3 == 0b011:
243             return "MULHU" if f7 == 0b0000001 else "SLTU"
244         return "ADD"
245

```

Alu_control Changes

Alu_exec Changes

```
268 def alu_exec(alu_op, a, b):
293     if alu_op == "MUL":
294         return u32(a * b)
295     if alu_op == "MULH":
296         product = s32(a) * s32(b)
297         return u32(product >> 32)
298     if alu_op == "MULHSU":
299         product = s32(a) * b
300         return u32(product >> 32)
301     if alu_op == "MULHU":
302         product = a * b
303         return u32(product >> 32)
304     if alu_op == "DIV":
305         if b == 0:
306             return 0xFFFFFFFF # -1 here
307         if s32(a) == -0x80000000 and s32(b) == -1: # Checking for overflow conditions
308             return a # Returns dividend
309         result = abs(s32(a)) // abs(s32(b))
310         if (s32(a) < 0) ^ (s32(b) < 0): # Checking if signs differ
311             result = -result
312         return u32(result)
313     if alu_op == "DIVU":
314         return u32(a // b) if b != 0 else 0xFFFFFFFF # 2^32 - 1 here
315     if alu_op == "REM":
316         if b == 0:
317             return u32(a)
318         if s32(a) == -0x80000000 and s32(b) == -1: # Checking for overflow conditions
319             return 0
320         result = abs(s32(a)) % abs(s32(b))
321         if s32(a) < 0: # matching sign of remainder to sign of dividend
322             result = -result
323         return u32(result)
324     if alu_op == "REMU":
325         return a % b if b != 0 else u32(a) # Return dividend
326
```

Sample Program

- Sample program for demonstrating all instructions added to RISC-V Simulator
- Using both unsigned ($x2 == 2$) and signed ($x1 == -7$) operators to show full functionality of instructions

```
1  addi x1, x0, -7
2  addi x2, x0, 2
3  mul  x10, x1, x2
4  mulh x11, x1, x2
5  mulhsu x12, x1, x2
6  mulhu x13, x1, x2
7  div  x14, x1, x2
8  divu x15, x1, x2
9  rem  x16, x1, x2
10 remu x17, x1, x2
```

References

RISC-V International. (2026). The RISC-V Instruction Set Manual Volume I Unprivileged Architecture. https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf